Depending on the nature of the request, the controller may take a variety of actions, but the key point is that the actions are undertaken using the application model. The model here contains all the business logic for the application, so it's able to provide requested data or perform actions.

NOTE In this description of MVC, the model is considered to be a complex beast, containing all the logic for how to perform an action, as well as any internal state. The Razor Page `PageModel` class is *not* the model we're talking about! Unfortunately, as in all software development, naming things is hard.

Consider a request to view a product page for an e-commerce application. The controller would receive the request and know how to contact some product service that's part of the application model. This service might fetch the details of the requested product from a database and return them to the controller.

Alternatively, imagine that a controller receives a request to add a product to the user's shopping cart. The controller would receive the request and most likely would invoke a method on the model to request that the product be added. Then the model would update its internal representation of the user's cart, by adding (for example) a new row to a database table holding the user's data.

TIP You can think of each Razor Page handler as being a mini controller focused on a single page. Every web request is another independent call to a controller that orchestrates the response. Although there are many controllers, all the handlers interact with the same application model.

After the model has been updated, the controller needs to decide what response to generate. One of the advantages of using the MVC design pattern is that the model representing the application's data is decoupled